

When run, the main code of the game instantiates all the scenes. In the Play scene, the `__init__()` method creates all the buttons and text display fields for the bottom of the window ❶, then loads the sounds. Very importantly, we use composition, discussed in Chapters 4 and 10, to create a Player object (`oPlayer`), a Baddie manager object (`oBaddieMgr`), and a Goodie manager object (`oGoodieMgr`) ❷. The Play scene object creates these managers and expects them to create and manage all the Baddies and Goodies. The `__init__()` method runs when the program starts, but doesn't actually start the game. Instead, it implements a state machine (as discussed in Chapter 15) that starts in the waiting state ❸. A round of the game starts when the user presses New Game.

All scenes must have a `getSceneKey()` method ❹ that returns a string representing the current scene. [Listing 16-4](#) shows the code that retrieves the scores and resets the game upon request.

```
❶ def enter(self, data):
    self.getHiAndLowScores()

❷ def getHiAndLowScores(self):
    # Ask the High Scores scene for a dict of scores
    # that looks like this:
    # {'highest': highestScore, 'lowest': lowestScore}
    ❸ infoDict = self.request(SCENE_HIGH_SCORES, HIGH_SCORES_DATA)
    self.highestHighScore = infoDict['highest']
    self.highScoreText.setValue(self.highestHighScore)
    self.lowestHighScore = infoDict['lowest']

❹ def reset(self): # start a new game
    self.score = 0
    self.scoreText.setValue(self.score)
    self.getHiAndLowScores()

    # Tell the managers to reset themselves
    ❺ self.oBaddieMgr.reset()
    self.oGoodieMgr.reset()

    if self.backgroundMusic:
        pygame.mixer.music.play(-1, 0.0)
    ❻ self.startButton.disable()
    self.highScoresButton.disable()
    self.soundCheckBox.disable()
```